

# CS213: Reverse Engineering Lab

Instructor: Dr Shreya Goyal

## Problem 1: Linear Regression Model Recovery

### Objective

Reverse engineer a linear regression model by probing it with test inputs and recovering the learned coefficients.

### Problem Statement

You are given a black-box linear regression model that takes 3 input features and produces a continuous output:

`model.predict(X) → y_pred`

The model was trained on data with an intercept term. Your task is to recover:

1. The intercept term
2. All three coefficients
3. An estimate of the model's MSE on training data

You can make up to 100 API calls to the model

You cannot inspect model weights directly

You can only query: **`model.predict(X)`** where `X` is a numpy array of shape `(n_samples, 3)`

### Lab Tasks

#### Task 1: Recover the Intercept

- Query the model with `X = [[0, 0, 0]]`
- Explain why this gives you the intercept

#### Task 2: Recover Individual Coefficients

- Use a systematic approach to find  $\beta_1, \beta_2, \beta_3$
- For each feature, vary it while holding others constant
- Record the change in output per unit change in input
- Explain the mathematical principle behind your approach

### Task 3: Estimate Model Error

- Generate random test points
- Measure the variance in predictions
- Estimate whether the model exhibits high bias or high variance

### Task 4: Detect Non-linearity (Bonus)

- Test if there's evidence the model is non-linear
- What patterns would indicate the model learned interactions?

## Expected Deliverables

Your solution should provide:

```
intercept = recover_intercept(model)
coefficients = recover_coefficients(model) ( shape (3,))
mse_estimate = estimate_model_error(model)
```

```
# Visualize decision surface if possible
plot_model_behavior(model)
```

Datasets: <https://archive.ics.uci.edu/dataset/165/concrete+compressive+strength>  
<https://www.kaggle.com/code/prasadperera/the-boston-housing-dataset>

## Problem 2: Logistic Regression Decision Boundary Discovery

Reverse engineer a binary classification model and visualize its decision boundary without access to model parameters.

### Problem Statement

You have a black-box logistic regression model with 2 features that classifies points as class 0 or class 1:

```
model.predict(X) → {0, 1}
model.predict_proba(X) → probability of class 1
```

Your task is to:

1. Find the decision boundary ( $P(\text{class } 1) = 0.5$ )
2. Estimate the model coefficients
3. Determine feature importance
4. Identify potential feature interactions

## Lab Tasks

### Task 1: Grid-Based Boundary Search

- Create a 2D grid of test points
- Query the model for each point
- Plot the regions where model predicts class 0 vs class 1
- Visualize the decision boundary

### Task 2: Recover Model Coefficients

- Use logistic regression theory:

$$P(y = 1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2)}}$$

- 
- Sample points along and near the boundary
- Solve for the coefficients using your samples
- Verify your estimates by comparing predictions

### Task 3: Feature Importance Analysis

- Compute feature importance by measuring prediction changes when perturbing each feature
- Which feature has more influence on the decision?
- Is there evidence of feature interactions?

### Task 4: Robustness Testing (Bonus)

- Test with adversarial examples near the decision boundary
- How confident is the model near the boundary?
- Identify regions where the model is most/least confident

## Expected Deliverables

- Visualizations:
  - `fig, axes = plt.subplots(2, 2, figsize=(12, 10))`
- Decision boundary heatmap
  - `plot_decision_boundary(model, ax=axes[0, 0])`
- Feature importance
  - `plot_feature_importance(model, ax=axes[0, 1])`
- Model confidence (probability) heatmap
  - `plot_confidence_heatmap(model, ax=axes[1, 0])`
- Recovered vs actual coefficients

```
plot_coefficient_recovery(estimated_coef, true_coef, ax=axes[1, 1])
```

- Metrics:  
print(f"Coefficient Recovery Error: {coefficient\_error:.4f}")  
print(f"Decision Boundary Accuracy: {boundary\_accuracy:.4f}")

Datasets:

<https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>

## Problem 3: Neural Network Layer-Wise Feature Extraction

### Objective

Reverse engineer what features a neural network learns at each layer by analyzing intermediate representations.

### Problem Statement

You have a black-box neural network with architecture:

- Input layer: 20 features
- Hidden layer 1: 8 neurons (ReLU)
- Hidden layer 2: 4 neurons (ReLU)
- Output layer: binary classification (sigmoid)

You can query intermediate layer outputs:

`model.predict(X)` → class prediction

`get_layer_output(X, layer_name)` → hidden layer activations

Your task is to:

1. Understand what each hidden layer learns
2. Identify which input features are most important
3. Determine if the network has learned meaningful feature combinations
4. Detect potential bottlenecks or redundancy

### Lab Tasks

#### Task 1: Input Feature Importance

- For each of the 20 input features, measure its impact on the final prediction
- Use gradient-based importance or perturbation-based importance
- Rank features by importance
- Visualize the top-10 most important features

#### Task 2: First Hidden Layer Analysis

- Extract activations from the first hidden layer for diverse input samples
- Visualize the 8 neurons using heatmaps or t-SNE

- Can you interpret what each neuron is detecting?
- Are some neurons redundant?

### Task 3: Second Hidden Layer Analysis

- Analyze the 4 neurons in the second hidden layer
- These should be more abstract representations
- Do they cluster class 0 and class 1 samples well?
- Visualize: can you create a 2D/3D projection of the second layer?

### Task 4: Feature Interaction Detection

- Some pairs of input features might interact
- Measure: what happens when you vary feature A while keeping others fixed?
- Then measure: what happens when you vary A while varying B?
- Do you see non-linear interactions?

## Expected Deliverables

Analysis functions:

```
input_importance = compute_input_importance(model)
first_layer_analysis = analyze_layer(model, layer_id=1)
second_layer_analysis = analyze_layer(model, layer_id=2)
interactions = detect_feature_interactions(model)
```

- Visualizations:
  - `fig = plt.figure(figsize=(16, 12))`
- Input feature importance bar chart
  - `ax1 = plt.subplot(2, 3, 1)`
  - `plot_feature_importance_bar(input_importance, ax=ax1)`
- First layer neuron activations (heatmap)
  - `ax2 = plt.subplot(2, 3, 2)`
  - `plot_layer_activations(first_layer_analysis, ax=ax2)`
- Second layer t-SNE with class labels
  - `ax3 = plt.subplot(2, 3, 3)`
  - `plot_hidden_layer_tsne(model, layer_id=2, ax=ax3)`
- Feature interaction heatmap
  - `ax4 = plt.subplot(2, 3, 4)`
  - `plot_interaction_matrix(interactions, ax=ax4)`
- Neuron contribution analysis
  - `ax5 = plt.subplot(2, 3, 5)`
  - `plot_neuron_contributions(model, ax=ax5)`
- Model complexity vs performance

```
ax6 = plt.subplot(2, 3, 6)
plot_layer_pruning_analysis(model, ax=ax6)
```

```
plt.tight_layout()
plt.savefig('network_analysis.png', dpi=150)
```

Dataset: <https://www.tensorflow.org/datasets/catalog/mnist>